

Fayda MCP Bridge

System architecture and coder implementation plan | 6 October 2026

Chosen stack

Python, FastAPI, standalone FastMCP, Neon PostgreSQL, Redis, and Fayda eSignet. Use a modular monolith: one application with explicit modules. Run a separate worker only when scheduled cleanup or notifications require it.

Product boundary

Businesses connect their own registered Fayda relying-party credentials. Agents call standardized MCP tools. People authenticate and consent in eSignet. The bridge verifies signed identity responses, evaluates permitted claims, and returns minimal verification results.

The bridge performs identity verification. Sector-specific KYC decisions, sanctions checks, and additional compliance rules belong in the integrating application or a separate policy workflow. Fayda identity alone does not prove citizenship.

Credential ownership

Credential	Owner and use
Bridge access token	Business or authorized operator. Grants access to the MCP resource.
Fayda client ID and signing key	Business relying party. Used only by the backend for eSignet client authentication.
Citizen authentication	Person verifying identity. Completed with eSignet. Never an agent tool input.
Bridge signing key	Bridge operator. Optional signing of bridge-issued result receipts, separate from Fayda keys.

Implementation assumptions

Confirm Fayda approval for your hosted credential-custody model, registered callback URLs, available claims, assurance levels, and sandbox/production endpoints before integration. A business supplies its client ID and required signing credentials, rather than a generic Fayda API key. This document proposes the bridge design and does not claim all providers expose identical claims.

1. Components and deployment

Component	Responsibility
HTTPS ingress	TLS, trusted hosts, request limits, safe forwarding, callback query redaction.
FastAPI application	REST onboarding, browser start/callback routes, health endpoints, shared application services.
FastMCP mount: /mcp	Streamable HTTP tools, bridge token validation, tool scopes, tenant authorization.
Verification service	Session lifecycle, idempotency, consent binding, result persistence.
Fayda adapter	Authorization Code + PKCE S256, private-key client assertions, token and UserInfo requests.
JWT / claim service	Trusted JWKS validation, identity response binding, deterministic claim evaluation.
Neon PostgreSQL	Durable tenant configuration, requests, permissions, results, audit records.
Redis	Expiring OIDC state, nonce, PKCE verifier, rate limits, brief JWKS cache.
Secrets vault / KMS	Per-business Fayda signing keys, key versions, rotation, least-privilege access.
OAuth authorization server	Issues bridge-audience access tokens. Separate from Fayda citizen authentication.

Deployment layout

Deploy one API container behind HTTPS. Connect to Neon and managed Redis over TLS. Add a worker container for durable outbox processing and expiry cleanup. Keep the OAuth authorization server and secrets store as separate trusted dependencies. Reuse a proven OAuth service rather than implementing an authorization server from scratch.

Mount only explicitly authored MCP tools. Do not convert admin routes or the complete OpenAPI schema into tools. Combine FastMCP and application lifespans so Redis, database, HTTP client, and MCP session management start and close correctly. Verify the final public /mcp path against your pinned FastMCP release.

2. Verification flow and trust boundaries

1. Business onboarding: an authorized administrator registers tenant metadata, approved Fayda client configuration, callback URL, signing-key reference, claim permissions, and purpose policies.
2. Agent calls `start_verification` with its bridge access token, purpose, approved claim checks, an opaque application user reference, and an idempotency key. Derive tenant and principal from the validated token.
3. Service creates a durable request in Neon. Return a short-lived opaque verification link and request ID. Bind the request to tenant, caller, and application user reference.
4. Person opens the link. Show business, purpose, requested information, and expiry. Require an explicit continue action before redirecting. Generate state, nonce, and PKCE verifier server-side.
5. Redis stores temporary state with a TTL. Browser receives a secure `HttpOnly` session cookie. Redirect to the configured eSignet authorization endpoint with approved scopes, claims, state, nonce, and S256 challenge.
6. Person authenticates and gives consent through eSignet. The bridge never receives OTPs, biometric captures, or biometric templates.
7. Callback atomically consumes state, validates browser binding and request status, and exchanges the code server-side using the business signing key and PKCE verifier. Never pick a tenant from an untrusted callback parameter.
8. Validate ID token and signed `UserInfo` using trusted provider keys. Check allowed algorithm, issuer, audience, expiry, nonce where applicable, and matching subject. Retrieve only requested claims.
9. Evaluate policy in Python. Derive `age_over_18` from a valid birthdate only if permitted. Treat missing or malformed claims as unavailable. Store the minimal result and audit events in Neon, then discard tokens and transient demographic values.
10. Agent polls `get_verification_result`. Enforce tenant, caller, and scope on every read. Return status, permitted checks, `verified_at`, expiry, assurance information if present, and evidence reference.

Cryptographic meaning

A validated Fayda signature authenticates the received payload. An age predicate computed by your service is a bridge-derived decision, not a Fayda-signed age assertion or a zero-knowledge proof. An optional bridge-signed receipt identifies the bridge as issuer and references the verification evidence.

3. MCP tools and HTTP routes

Tool	Inputs and outcome
start_verification	purpose, checks, application_user_ref, idempotency_key. Returns request_id, verification_url, expires_at.
get_verification_status	request_id. Returns pending, processing, verified, rejected, failed, expired, or cancelled.
get_verification_result	request_id. Returns only approved checks and evidence metadata after completion.
cancel_verification	request_id. Cancels an unfinished request owned by the caller. Optional for MVP.

Scopes: verification:create, verification:read, verification:cancel. Tenant policy further restricts purpose and claim names. Never accept private keys, OTPs, biometric data, tokens, arbitrary provider URLs, or a trusted tenant_id as tool arguments.

FastAPI routes

```
POST /api/v1/admin/fayda-connections
GET /api/v1/admin/fayda-connections/{id}
POST /api/v1/admin/fayda-connections/{id}/rotate-key
GET /verify/{opaque_link} # purpose and consent page
POST /verify/{opaque_link}/start # CSRF-protected redirect
GET /auth/fayda/callback # registered provider callback
GET /health/live
GET /health/ready
/mcp # Streamable HTTP MCP
/.well-known/oauth-protected-resource # MCP auth metadata
```

Admin routes require a separate administrative identity and role. Responses never return private signing material. Callback and citizen routes use browser/OIDC protections rather than requiring an agent token. Do not authorize the entire application using one MCP-only middleware rule.

Example result

```
{
  "request_id": "vr_opaque",
  "status": "verified",
  "checks": {"identity_verified": true, "age_over_18": true},
  "verified_at": "2026-10-06T13:00:00Z",
  "expires_at": "2026-10-06T13:15:00Z",
  "evidence_ref": "ev_opaque",
  "policy_version": "v1"
}
```

identity_verified means successful authentication under the approved policy. It does not establish ownership of an existing application account by itself. Bind the verification link to the intended application session or perform an explicit account-linking confirmation.

4. FastAPI folders and files

```
fayda-mcp-bridge/
app/
  main.py                # FastAPI factory and MCP mount
  lifespan.py            # DB, Redis, HTTP, MCP lifecycle
  core/
    config.py            # Pydantic settings
    auth.py              # bridge token validation
    permissions.py       # tenant, tool, purpose checks
    logging.py           # structured logs and redaction
    errors.py            # safe domain error mapping
  api/
    router.py
    routes/
      onboarding.py      # admin tenant/Fayda configuration
      verification.py     # human verification/start routes
      callback.py         # eSignet callback
      health.py
    dependencies.py      # services and admin principal
  mcp/
    server.py            # standalone FastMCP instance
    tools.py             # thin typed tool handlers
    auth.py              # OAuth resource/token integration
  schemas/
    onboarding.py
    verification.py
    results.py
  services/
    verification.py      # lifecycle orchestration
    claims.py            # deterministic predicates
    policy.py            # permitted purpose/claims
    consent.py           # purpose and granted claims
    audit.py             # safe audit events
    receipts.py          # optional bridge-signed receipts
  integrations/fayda/
    client.py            # bounded async HTTP calls
    oidc.py              # redirect and code exchange
    assertions.py        # client assertion signing
    tokens.py            # signed response validation
    jwks.py              # trusted keys and rotation
    mapping.py           # claim/localization normalization
  db/
    session.py           # async SQLAlchemy + psycopg
    models.py            # tenant-scoped tables
    repositories.py      # tenant-enforced queries
  infra/
    redis.py             # sessions, TTL, rate limits
    secrets.py           # secret-store interface
  workers/
    cleanup.py           # expiry and retention jobs
  templates/
    verify.html
    complete.html
  migrations/            # Alembic schema history
  tests/
    unit/               # policy, mappings, JWT rejection
    integration/         # callback, tenancy, real stores
    contract/            # MCP client and mocked provider
.env.example
.gitignore
pyproject.toml
uv.lock
alembic.ini
Dockerfile
compose.yaml            # local Redis and API
README.md
```

Tools and routes call the same service layer. Repositories own SQL access. Fayda modules never trust decoded JWT payloads until signature and context validation succeed. Pin dependencies and commit the lockfile.

5. Environment variables

Commit `.env.example` with placeholders. Keep `.env` ignored. Production receives secrets through the deployment secret manager. The values below are project-defined settings, not built-in FastMCP variable names.

```
APP_ENV=development
APP_NAME=fayda-mcp-bridge
PUBLIC_BASE_URL=https://bridge.example.com
LOG_LEVEL=INFO
ALLOWED_HOSTS=["bridge.example.com", "localhost"]
CORS_ORIGINS=["https://portal.example.com"]

# SQLAlchemy psycopg URLs. URL-encode username and password.
DATABASE_URL=postgresql+psycopg://USER:PASS@HOST-pooler/DB?sslmode=require
DATABASE_MIGRATION_URL=postgresql+psycopg://USER:PASS@HOST/DB?sslmode=require
DB_POOL_SIZE=5
DB_MAX_OVERFLOW=5

REDIS_URL=rediss://default:PASSWORD@HOST:6379/0
REDIS_KEY_PREFIX=fayda_bridge:dev:
OIDC_SESSION_TTL_SECONDS=600
VERIFICATION_LINK_TTL_SECONDS=600
RESULT_TTL_SECONDS=900
RATE_LIMIT_PER_MINUTE=30

# Bridge OAuth resource, independent of Fayda
MCP_RESOURCE_URL=https://bridge.example.com/mcp
MCP_AUTH_ISSUER=https://auth.example.com/
MCP_AUTH_JWKS_URL=https://auth.example.com/.well-known/jwks.json
MCP_REQUIRED_SCOPES=["verification:create", "verification:read"]

# Provider endpoints supplied/confirmed during Fayda onboarding
FAYDA_ENVIRONMENT=sandbox
FAYDA_ISSUER_URL=https://REPLACE_APPROVED_ISSUER
FAYDA_AUTHORIZATION_URL=https://REPLACE_APPROVED_AUTH_ENDPOINT
FAYDA_TOKEN_URL=https://REPLACE_APPROVED_TOKEN_ENDPOINT
FAYDA_USERINFO_URL=https://REPLACE_APPROVED_USERINFO_ENDPOINT
FAYDA_JWKS_URL=https://REPLACE_APPROVED_JWKS_ENDPOINT
FAYDA_REDIRECT_URI=https://bridge.example.com/auth/fayda/callback
FAYDA_ALLOWED_ALGORITHMS=["RS256"]
FAYDA_HTTP_TIMEOUT_SECONDS=10
FAYDA_CLIENT_ASSERTION_TTL_SECONDS=120

SECRETS_BACKEND=vault
SECRETS_BASE_URL=https://vault.example.com
SECRETS_NAMESPACE=fayda-bridge
# Prefer workload identity over a long-lived vault token.
AUDIT_RETENTION_DAYS=90
RESULT_RETENTION_DAYS=30
JWT_CLOCK_SKEW_SECONDS=60
JWKS_CACHE_TTL_SECONDS=300
```

TTL, rate-limit, and retention values are initial engineering defaults. Confirm them with business requirements and approved retention policy. A result expiry limits reuse, while retention controls later deletion. Use provider-approved algorithms and assertion lifetime.

Per-tenant client IDs, key references, scopes, claims, and approved callback configuration belong in Neon. Private keys belong in the secrets store. Do not use one global `FAYDA_CLIENT_ID` or private key for all businesses. Provider URLs come from an operator-approved environment catalog, never arbitrary agent input.

6. Neon schema and Redis design

Neon table	Key fields
tenants	id, name, status, created_at
principals	id, tenant_id, oauth_subject, roles, status
fayda_connections	id, tenant_id, environment, client_id, secret_ref, key_version, config_version, allowed_scopes, allowed_claims, status
purpose_policies	id, tenant_id, purpose, required_checks, permitted_claims, required_assurance, version
verification_requests	id, tenant_id, principal_id, application_user_ref, connection_id, config_version, purpose, checks, status, expires_at, idempotency_key
consent_records	request_id, tenant_id, purpose, requested_claims, granted_claims, consent_recorded_at, source
verification_results	request_id, tenant_id, minimal_checks, assurance_metadata, verified_at, expires_at, evidence_ref, policy_version
audit_events	id, tenant_id, request_id, actor_ref, event_type, timestamp, safe_metadata
outbox_events (optional)	id, tenant_id, request_id, event_type, delivery_state, attempts

Use UUIDs and explicit tenant_id filters. Add composite foreign keys where practical to prevent cross-tenant references. Unique constraint: tenant_id + principal_id + idempotency_key. Store a keyed HMAC of provider subject only if future identity correlation is required, scoped to tenant. Do not store Fayda numbers by default.

Redis records

oidc:state:<hashed_state>: tenant, caller, request, nonce, PKCE verifier, browser binding, selected configuration version. TTL 10 minutes. Consume atomically and mark the Neon request processing to reject duplicate callbacks.

link:<hashed_handle>: request reference and short-lived browser-flow binding.

rl:<tenant>:<principal>:<window>: rate counters. jwks:<approved_issuer>: provider public keys with bounded TTL. Do not treat any opaque handle as authorization to retrieve results.

Neon is the source of truth. Redis loss expires temporary sessions and requires a fresh verification. Do not reconstruct consumed state or mark a request verified after Redis failure. Reject new starts while required session storage is unavailable.

Use Neon pooled runtime connections and a direct migration connection. Keep per-process SQL pools small and account for replicas. Use TLS, separate runtime and migration roles, and rollback-capable migrations. Do not depend on session-level tenant variables across pooled transactions.

7. Classified tasks: foundation and access

A1 | Project foundation

Build: Create the folder layout, Pydantic settings, lockfile, FastAPI factory, explicit FastMCP tools, combined lifespans, Docker image, and local Redis setup.

Acceptance: The server boots, exposes /mcp and health routes, and shuts down every client cleanly.

A2 | Neon persistence

Build: Implement SQLAlchemy async sessions with psycopg, Alembic migrations, tenant-aware repositories, indexes, and idempotency constraints.

Acceptance: A migration runs on a disposable database. Cross-tenant reads and writes fail. Duplicate starts return the existing request.

A3 | Redis primitives

Build: Implement TLS connection, random handles, atomic state consumption, TTLs, browser bindings, and per-principal rate limits.

Acceptance: Expired, reused, or mismatched state is rejected. Redis failure never produces a verified result.

B1 | Bridge OAuth authorization

Build: Integrate a proven authorization server. Expose protected-resource metadata and validate token issuer, signature, audience, expiry, and scopes.

Acceptance: A real MCP client discovers authorization and calls an allowed tool. Tokens for Fayda or another resource fail.

B2 | Tenant and principal permissions

Build: Resolve tenant from trusted OAuth identity and local mapping. Enforce tenant, principal, purpose, and claim policy on every tool and repository query.

Acceptance: An agent with a valid token cannot access another principal or tenant request. Tool visibility alone never grants access.

B3 | Business Fayda onboarding

Build: Build admin routes for client ID, environment, registered callback, allowed claims, and signing-key import/reference. Validate ownership and key format.

Acceptance: Admin credentials are required. Private keys never appear in API responses, database rows, logs, or MCP arguments.

B4 | Key lifecycle

Build: Support secret versions, rotation, connection suspension, and config snapshots per request. Use a supported JWK import library.

Acceptance: New requests use the new configuration. In-flight requests retain compatible configuration or expire safely. Disabling a tenant blocks requests.

8. Classified tasks: identity and tool behavior

C1 | Human start and OIDC redirect

Build: Create the purpose page and CSRF-protected start action. Generate state, nonce, and PKCE S256. Request only approved claims.

Acceptance: Browser binding survives the callback. Link scanners do not initiate a flow. The business and requested purpose are visible.

C2 | Provider client authentication

Build: Implement `private_key_jwt` client assertions with client ID as issuer/subject, token endpoint audience, short expiry, unique jti, and approved algorithm. Exchange the code with its original PKCE verifier.

Acceptance: Mock and sandbox flows succeed. Incorrect PKCE or signing key fails without credential leakage.

C3 | Signed response verification

Build: Verify ID token and signed `Userinfo` with trusted provider JWKS. Check issuer, audience, expiry, issued-at bounds, nonce in ID token, azp where relevant, and matching sub.

Acceptance: Unsigned, forged, expired, wrong-audience, wrong-issuer, and mismatched-subject responses all fail. Unknown key refresh is bounded.

C4 | Claim evaluation and consent

Build: Map language-specific fields, validate date formats, implement permitted identity/age checks, record actual granted fields separately from requested fields, and enforce assurance policy.

Acceptance: Missing DOB produces unavailable, never a fabricated false or true. Declined optional claims stay unavailable. Required missing claims prevent the requested check.

D1 | MCP tools

Build: Implement typed tool inputs, safe result schemas, policy enforcement, tenant/caller binding, short result expiry, and tool error mapping.

Acceptance: MCP contract tests execute start, pending status, completed result, expiry, and denial flows. Raw tokens and profile data stay out of responses.

D2 | Request lifecycle

Build: Implement `pending` -> `processing` -> `verified/rejected/failed` and `pending` -> `expired/cancelled`. Use conditional updates to prevent concurrent finalization.

Acceptance: Only validated identity responses reach `verified`. Two callbacks never finalize twice. Repeated starts respect idempotency.

D3 | Failure handling

Build: Distinguish consent decline, authentication failure, provider outage, expired state, unavailable claims, and policy rejection. Avoid blind code-exchange retries after ambiguous network failures.

Acceptance: Caller receives safe errors. A provider outage leaves a failed or indeterminate request requiring a new flow, never a successful verification.

9. Classified tasks: operations and release

E1 | Privacy and audit

Build: Implement an allowlist for log fields, query-string redaction, safe error telemetry, request events, consent records, and retention cleanup.

Acceptance: Logs contain request IDs and outcome codes without OTPs, keys, JWTs, authorization codes, or demographic payloads.

E2 | Operational controls

Build: Add liveness/readiness, provider timeout metrics, callback latency, failure counts, rate-limit metrics, Redis health, Neon pool usage, and a runbook.

Acceptance: Alerts identify outages without exposing identity information. Readiness detects required storage failure. Liveness does not restart on provider outage.

E3 | Security and integration tests

Build: Run real Redis/SQL tests plus mocked OIDC/JWKS tests. Cover replay, browser session mismatch, key rotation, tenant isolation, revoked tenant, denied purpose, SSRF URL attempts, and secret redaction.

Acceptance: All authorization and signature rejection tests pass. A standard MCP client connects through the full OAuth flow.

E4 | Deployment and recovery

Build: Configure HTTPS, restricted network access, secret-store identity, safe proxy headers, backups, migrations, CI checks, and dependency updates. Keep synchronous verification on the API service.

Acceptance: Deploy and rollback succeed. Restore a database backup in a test environment. Cleanup jobs do not delete active requests.

E5 | Fayda sandbox and production gate

Build: Confirm registered relying parties, custody permissions, endpoints, callback registrations, PKCE, claims, JWKS, algorithms, assertion lifetime, and approved assurance modes.

Acceptance: A real sandbox authentication completes with signed validation. Production uses approved credentials and endpoints, with no sandbox key reuse.

Build order

A1-A3 first. Then B1-B4. Next C1-C4, followed by D1-D3. Complete E1-E5 before production. Implement one identity check end to end before adding age checks, optional receipts, webhooks, or queues.

MVP completion

One onboarded business uses its own Fayda credentials. A permitted agent starts verification. A person authenticates and consents. Signed responses pass validation. The correct tenant receives a minimal result. Replay, cross-tenant access, forged tokens, and expired sessions fail.

10. Source references and implementation decisions

Official documentation checked on 6 October 2026. The architecture, file names, database schema, environment variable names, and task grouping are proposed design decisions.

Fayda relying-party integration

Supports the provider-specific authorization-code/PKCE flow and client-ID/private-key credential model. Confirm deployment configuration with onboarding. This plan requires signature validation even where illustrative provider snippets only decode a JWT. Never copy `verify_signature=False` into verification logic.

<https://nidp.atlassian.net/wiki/spaces/~712020e898f1e6ab3543aabe17e0865c3305f5/pages/845905921>

FastMCP HTTP deployment and FastAPI integration

Use the pinned release documentation for mount paths, authentication wiring, and lifecycle composition. Standalone FastMCP uses the `fastmcp` package.

<https://gofastmcp.com/integrations/fastapi>

<https://github.com/PrefectHQ/fastmcp/blob/main/docs/deployment/http.mdx>

MCP authorization and security

Bridge authorization is a separate OAuth resource boundary. Enforce audience validation, protected-resource discovery, and caller-bound handles. Never forward the agent access token to Fayda.

<https://modelcontextprotocol.io/specification/latest/basic/authorization>

https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices

Neon and database driver

Use a TLS connection string from Neon, pooled runtime access, and a migration connection with suitable privileges. `Psycopg` supports async SQLAlchemy usage.

<https://neon.com/docs/connect/connection-pooling>

<https://neon.com/docs/get-started/connect-neon>

<https://docs.sqlalchemy.org/en/20/dialects/postgresql.html>

Explicit exclusions from the first release

No autonomous biometric collection. No agent-facing credential submission. No arbitrary citizen lookup without a consented flow. No citizenship inference from Fayda possession. No zero-knowledge proof claims. No compliance approval based only on successful authentication.